Приднестровский государственный университет им. Т.Г. Шевченко
Физико-технический институт

ПРАКТИЧЕСКИЙ КУРС ВЕБ РАЗРАБОТКИ REACT TYPESCRIPT NODE И SUPABASE

Лабораторный практикум

Разработал:
ст. преподаватель
кафедры ИТ
Бричаг Д.В.

г. Тирасполь
2026 г.

Лабораторная работа 1

Старт проекта на React и TypeScript

В этой работе вы создадите каркас приложения, которое будете развивать весь семестр. Вы выберете тему, запустите React-проект, замените стартовый экран своим и опубликуете исходники в Git-репозитории. Следующая лабораторная добавит в этот же проект страницы, компоненты и навигацию.

Цель работы

Подготовить воспроизводимую основу приложения на React + TypeScript + Vite и освоить ежедневный цикл разработки: изменить код, проверить результат в браузере, собрать проект и сохранить изменения в Git.

Обязательный результат

Работающий начальный экран выбранного приложения, успешная проверка типов и сборка, удалённый Git-репозиторий и README с понятными командами запуска. Другой человек должен суметь скачать исходники и запустить проект по вашей инструкции.

Сейчас достаточно названия, описания и пустого раздела записей. Формы, маршрутизация, Supabase, авторизация и Node API появятся позже. На GitHub вы публикуете исходники; публичный URL приложения будет итогом курса.

Теоретическая справка

Как связаны технологии проекта

React описывает интерфейс через компоненты. В этой работе компонент App — функция, возвращающая JSX. Эта разметка похожа на HTML: значение переменной вставляют в фигурных скобках, а CSS-класс задают через className.

TypeScript добавляет проверку типов к JavaScript: переменная типа string должна содержать строку. Файлы с TypeScript и JSX имеют расширение .tsx. Инструменты проекта преобразуют исходники в JavaScript для браузера. Типы не заменяют проверку данных из сети.

Vite запускает сервер разработки, например на http://localhost:5173, и обновляет страницу при изменении кода. Отдельная команда build подготавливает production-сборку в frontend/dist.

Node.js на этом этапе нужен для запуска инструментов разработки. npm устанавливает зависимости и выполняет команды из package.json. Отдельный backend на Node.js будет создан в лабораторной 8.

Git сохраняет историю версий локально. GitHub хранит удалённую копию репозитория. Команда commit создаёт локальную версию, а push отправляет её на сервер. Один репозиторий будет использоваться для всех десяти лабораторных.

Какие файлы нужно понимать

frontend/index.html — HTML-точка входа с контейнером для React. frontend/src/main.tsx запускает React и подключает App. frontend/src/App.tsx содержит начальный компонент. Файлы CSS задают оформление. В index.html сохраняются контейнер root и подключение main.tsx.

package.json описывает зависимости и команды. package-lock.json фиксирует установленные версии и включается в Git. Каталог node_modules содержит установленные пакеты и в Git не включается: его восстанавливают командой npm ci. Файлы tsconfig задают проверку TypeScript, vite.config.ts — настройки Vite.

Практическая часть

Шаг 1 Выберите тему приложения

Выберите одну тему из списка. Можно предложить свою, если она позволяет выполнить тот же объём курса: основная сущность, связанные данные, личные аккаунты, CRUD, фильтры, серверная сводка и публикация.

1. Task Tracker — задачи, статусы, сроки и приоритеты.
2. Mini Issue Tracker — bugs/tasks и комментарии.
3. Expense Tracker — доходы, расходы и категории.
4. Каталог фильмов и сериалов — просмотр, оценки и избранное.
5. Каталог игр — платформы и статусы прохождения.
6. Планировщик мероприятий — события и регистрация на них.
7. Habit Tracker — ежедневные привычки и отметки.
8. Учебный менеджер — задания, курсы и дедлайны.
9. Личная библиотека — книги, статусы чтения и заметки.
10. Personal CRM — контакты и история взаимодействий.

Запишите название, пользователя и основной сценарий в 2–3 предложениях. Например: «Task Tracker помогает студенту вести личные задачи. Пользователь добавляет задачу, указывает срок, меняет статус и находит просроченные записи». Этот пример не обязывает выбирать Task Tracker.

Шаг 2 Подготовьте рабочую среду

Понадобятся редактор кода, современный браузер, Git, аккаунт GitHub и Node.js с npm. Установите версию Node, согласованную для курса и совместимую с текущим Vite. Если генератор сообщает о несовместимости runtime, обновите Node до подходящей поддерживаемой версии и перезапустите терминал.

В терминале проверьте установку. Каждая команда должна вывести номер версии:

```
node --version
npm --version
git --version
```

Зафиксируйте фактическую версию Node в README и в файле .node-version в корне проекта, одной строкой без префикса v. Не копируйте произвольный номер версии из чужого проекта.

Шаг 3 Создайте проект на Vite

Создайте общую папку проекта, например course-app. Все команды вводите по одной строке. Внутри будет пакет frontend; папки backend и supabase добавятся позже. Отдельные проекты lab1, lab2 и так далее создавать не нужно.

```
mkdir course-app
cd course-app
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm run dev
```

При запросе установки create-vite подтвердите её. Если генератор предлагает дополнительные возможности, выберите обычный шаблон React с TypeScript. Откройте адрес, напечатанный сервером. Если порт 5173 занят, Vite может показать другой порт: используйте именно его.

Оставьте сервер работающим. Для других команд откройте второй терминал или остановите сервер сочетанием Ctrl+C. Для повторного запуска достаточно перейти в frontend и выполнить npm run dev.

Шаг 4 Соберите свой начальный экран

Замените содержимое frontend/src/App.tsx. Ниже приведён минимальный пример: адаптируйте название, описание и раздел списка под свою тему.

```
import './App.css'

const appTitle: string = 'Task Tracker'

export default function App() {
  return (
    <main className="app">
      <header>
        <h1>{appTitle}</h1>
        <p>Личные задачи, сроки и прогресс.</p>
      </header>
      <section aria-labelledby="items-title">
        <h2 id="items-title">Мои задачи</h2>
        <p>Здесь появится список ваших задач.</p>
      </section>
    </main>
  )
}
```

Замените стили frontend/src/App.css простыми правилами:

```
.app {
  max-width: 960px;
  margin: 0 auto;
  padding: 32px 20px;
}

.app section { margin-top: 32px; }
```

Стартовый frontend/src/index.css тоже замените, чтобы демонстрационные настройки центрирования и тёмной темы не влияли на ваш экран:

```
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #202020;
  background: #ffffff;
}
```

Сохраните импорт index.css в main.tsx. В frontend/index.html укажите lang="ru" и название своего приложения в title. Удалите подключение стандартной иконки Vite, неиспользуемые логотипы и их импорты. Проверьте, что счётчика и ссылок стартового шаблона больше нет.

Добавьте собственное оформление: например, измените цвет заголовка и отступы. Текст должен читаться на узком экране без горизонтальной прокрутки. Кнопка добавления записи пока не нужна: этот сценарий появится вместе с формами.

Шаг 5 Проверьте структуру проекта

К концу работы структура должна выглядеть примерно так. Конфигурационные файлы генератора сохраняйте; перечень tsconfig может отличаться между версиями шаблона.

```
course-app/
  .gitignore
  .node-version
  README.md
  docs/
    progress.md
  frontend/
    package.json
    package-lock.json
    index.html
    vite.config.ts
    tsconfig.json
    tsconfig.app.json
    tsconfig.node.json
    src/
      main.tsx
      App.tsx
      App.css
      index.css
```

Папки pages, services и components появятся вместе с кодом следующих работ. README внутри frontend замените ссылкой на общую инструкцию в корне.

Шаг 6 Подготовьте README и историю этапов

Создайте README.md в корне. Это инструкция к приложению на весь семестр. Укажите название и тему, автора и группу, основной сценарий, стек, версию Node, команды запуска и то, что работает сейчас. В разделе текущих ограничений поясните, что данные и формы будут добавлены позже.

Команды в README должны учитывать папку frontend. Пример раздела запуска из корня уже скачанного проекта:

````
## Запуск
Из корня скачанного проекта:
```bash
cd frontend
npm ci
npm run dev
```

## Проверка сборки
Остановите dev через Ctrl+C. Выполните в frontend:
```bash
npm run build
npm run preview
```
````

Создайте docs/progress.md: кратко опишите результат ЛР 1 и 3–6 шагов проверки. При сдаче приложите идентификатор коммита, который можно посмотреть командой git log -1 --oneline.

Шаг 7 Сохраните проект в Git

Вернитесь из frontend в корень course-app командой `cd ..`. Здесь инициализируйте единственный репозиторий. Если Git уже инициализирован в корне, повторно создавать его не требуется.

```
cd ..
git init
```

Создайте корневой .gitignore. Эти правила действуют и на одноимённые каталоги вложенных пакетов:

```
node_modules/
dist/
.env
.env.*
!.env.example
*.local
```

Сохраните полезные правила .gitignore, созданного Vite внутри frontend. Реальные переменные окружения сейчас не нужны. Позже шаблон .env.example будет храниться в Git, а локальные значения — исключаться. Если файл уже попал в коммит, одно добавление в .gitignore не удалит его из истории.

Проверьте изменения перед фиксацией. При первом использовании Git задайте имя и email автора через git config user.name и git config user.email с вашими значениями.

```
git status
git add .
git diff --cached --stat
git commit -m "Lab 1: create React TypeScript app"
```

Создайте пустой репозиторий на GitHub, без автоматического README, .gitignore и лицензии. Скопируйте его HTTPS-адрес. В следующей команде замените YOUR_LOGIN и YOUR_REPOSITORY своими значениями:

```
git remote add origin https://github.com/YOUR_LOGIN/YOUR_REPOSITORY.git
git branch -M main
git push -u origin main
```

Если origin уже существует, сначала посмотрите git remote -v и используйте нужный адрес. Для закрытого репозитория предоставьте доступ преподавателю. В браузере убедитесь, что на GitHub есть исходники, README и lock-файл, а node_modules и dist отсутствуют.

Шаг 8 Проверьте типы и готовую сборку

В frontend/package.json найдите scripts.build. Он должен запускать TypeScript перед Vite. Для стандартного шаблона с несколькими tsconfig используется следующая команда:

```
"build": "tsc -b && vite build"
```

Сохраните strict: true в конфигурации TypeScript. Временно присвойте переменной appTitle число вместо строки и выполните npm run build: проверка типов должна завершиться ошибкой. Верните корректную строку и убедитесь, что сборка снова проходит. В Git сохраняется исправленный вариант.

```
cd frontend
npm run build
npm run preview
```

Откройте адрес preview из терминала и проверьте собственный экран. Preview показывает локально готовую сборку; он не публикует приложение в интернете. Остановить процесс можно через Ctrl+C. Открывать index.html двойным щелчком вместо сервера Vite не следует.

Шаг 9 Проверьте запуск из чистой копии

После отправки всех изменений остановите dev/preview. Перейдите в родительскую папку course-app и клонируйте репозиторий в новую, ещё не существующую папку course-app-check. Замените адрес в команде своим:

```
git clone https://github.com/YOUR_LOGIN/YOUR_REPOSITORY.git course-app-check
cd course-app-check/frontend
npm ci
npm run build
npm run dev
```

Если инструкция сработала без переноса файлов из исходной папки, проект воспроизводим. Если пришлось что-то исправить, внесите исправление в основной проект, закоммитьте и отправьте его; затем повторите проверку с актуальной версией. Продолжайте следующие лабораторные в основном рабочем каталоге.

Использование ИИ при выполнении работы

ИИ можно использовать для объяснения файлов, поиска причины ошибки и обсуждения небольших изменений. Например: «Объясни путь от index.html через main.tsx до App.tsx в моём проекте» или «Разбери эту ошибку TypeScript и предложи минимальное исправление». Передавайте текст ошибки и относящийся к ней фрагмент кода.

Проверяйте предложенное изменение в браузере и сборкой. Вы должны понимать свой App.tsx, уметь изменить его без повторной генерации проекта и объяснить команды. Использование ИИ необязательно и само по себе не приносит баллы. Если вы его использовали, кратко отметьте в README, с чем он помог и что вы проверили самостоятельно.

Частые проблемы

Команда node или npm не найдена. Проверьте установку Node и перезапустите терминал. Если PowerShell блокирует npm.ps1, используйте npm.cmd вместо npm.

Не найден package.json. Выполняйте команды npm из frontend. Ошибка импорта после удаления логотипа: уберите оставшуюся ссылку на файл.

Страница пустая. Проверьте первую ошибку в терминале и консоли браузера, экспорт App и наличие root в index.html.

package.json и lock-файл не совпадают. Выполните npm install в основном проекте, проверьте сборку и отправьте оба файла в Git. Повторите npm ci в актуальной проверочной копии.

git push не проходит. Проверьте origin, вход в GitHub и права доступа. После исправления убедитесь, что последний коммит появился на GitHub.

Результаты работы и сдача

Предоставьте преподавателю ссылку на репозиторий и коммит ЛР 1. В репозитории должны быть исходники React и TypeScript, CSS, конфигурация Vite и TypeScript, package.json и lock-файл, .gitignore, .node-version, README и docs/progress.md. Отдельный отчёт со скриншотами не требуется.

На демонстрации запустите приложение, покажите свой экран, измените его название или описание и объясните, где находится этот код. Покажите успешную сборку, содержимое репозитория и запуск из чистой копии. Преподаватель должен иметь доступ к исходникам.

Критерии оценки

Максимум — 10 баллов. Работа завершена, когда выполнен обязательный результат, а проект запускается по инструкции.

Обязательный функционал — 5 баллов: проект React + TypeScript + Vite запускается (2); экран соответствует теме и содержит название, описание и заглушку списка (2); проверка типов и сборка проходят (1).

Ошибки и проверка результата — 2 балла: нет ошибок приложения в терминале и консоли, нет оставшихся импортов удалённых ресурсов (1); студент продемонстрировал ошибку типов, исправление и просмотр готовой сборки (1).

Непрерывность проекта — 1 балл: выбрана тема на семестр, сформулирован основной сценарий, создан единый репозиторий с frontend и записью о первом этапе.

Git и воспроизводимость — 2 балла: актуальные исходники и lock-файл отправлены в доступный преподавателю репозиторий, лишние каталоги исключены (1); README и версия Node позволяют запустить чистую копию через npm ci и собрать её (1).

Вопросы для защиты

1. Как связаны index.html, main.tsx и App.tsx?
2. Чем JSX отличается от обычной HTML-разметки?
3. Какую ошибку в вашем проекте обнаруживает TypeScript?
4. Чем отличаются dev, build и preview?
5. Зачем нужен package-lock.json и почему node_modules не хранится в Git?
6. Чем отличаются commit и push?
7. Что уже работает в вашем приложении и что добавит ЛР 2?

Материалы для самостоятельной работы

Vite — создание проекта и шаблоны: https://vite.dev/guide/

React — TypeScript в компонентах: https://react.dev/learn/typescript

Vite — сборка и предварительный просмотр: https://vite.dev/guide/static-deploy

GitHub — исключение файлов из Git: https://docs.github.com/en/get-started/git-basics/ignoring-files